MSDS 451 Programming Assignment 1 Report

Predicting the Direction of Microsoft ($MSFT) Daily Returns

Prepared by Ashwin Senthilkumar

October 11, 2026

Problem Description

The objective of this project is to predict whether Microsoft ($MSFT) will close higher or lower than the day before using only information available at the end of the previous day. I chose MSFT because I work there and have been following the stock for the last four years. Microsoft is a heavily traded stock with more than 25 years of daily trading history and substantial trading activity, so there is plenty of data.

We frame this as binary classification. The target is assigned a value of 1 when the daily log return is positive (the price closed higher than the day before) and 0 when the return is zero or negative. The predictors consist exclusively of lagged prices, daily price ranges, lagged trading volumes, and moving averages calculated from previous closing prices. This design follows the course framework: features are created with Python Polars, subsets are selected using all-possible-subsets logistic regression scored by Akaike Information Criterion (AIC), an XGBoost classifier, and the final model is evaluated with a confusion matrix, a classification report and an ROC curve.

A model that can predict stock direction even slightly correctly could support systematic trading, but market efficiency suggests that consistently identifying these patterns is difficult. The real question is how much better than random guessing the model can perform on data it has not seen, and this remains the focus in this analysis.

Data Preparation and Pipeline

Source and sample. Daily open, high, low, close, and volume data for MSFT came from Yahoo! Finance using the yfinance package (getdata_yfinance.py). The dataset spans January 3, 2000 to September 24, 2025, and is stored in msds_getdata_yfinance_msft.csv. We dropped the Dividends and Stock Splits columns because they were not contributing to this prediction task. Since Yahoo! Finance adjusts prices for splits and dividends, the historical price series remains consistent across MSFT's previous stock splits.

Feature Engineering. Using Polars, we created fifteen candidate features (Table 1). Daily high-minus-low (HML) and open-minus-close (OMC) values describe how wide the day's price range was and the difference between opening and closing prices. These kinds of patterns are the ones technical analysts read from candlestick charts. HML, OMC, closing price, and trading volume each get lagged by one, two, and three days. We also calculated three exponential moving averages of the closing price (half-lives of 1, 2, and 4 days) labeled EMA2, EMA4, and EMA8. These averages are computed from the one-day-lagged closing price, so they exclude information from the day being predicted.

Table 1. The 15 candidate features, indexed from 0 to 14 as shown in the program output.

#	Feature	Description
0 to 2	CloseLag1 to 3	Closing price, lagged one to three days
3 to 5	HMLLag1 to 3	High minus low price, lagged one to three days
6 to 8	OMCLag1 to 3	Open minus close price, lagged one to three days
9 to 11	VolumeLag1 to 3	Trading volume, lagged one to three days
12 to 14	CloseEMA2, 4, 8	Exponential moving averages of the lagged close

Cleaning and Target. Because the earliest observations have no lagged values, the rows with nulls were dropped. This resulted in 6,468 daily observations from 01-06-2000 to 09-24-2025. The target classes are relatively balanced, with 3,307 days having positive returns (51.1%) and 3,161 days had flat or negative returns (48.9%). It also gives a simple benchmark for comparison: A model that always predicts an increase would be right about 51 percent, which a useful model should exceed. All computed features were written to msft-with-computed-features.csv so the calculations could be reviewed in Excel.

Scaling. We standardized the features (zero mean, unit variance) for the logistic regressions for subset selection. Since XGBoost does not depend on feature scaling, the final model was trained using the selected features in their unscaled, original form.

Feature Selection

The objective was to identify a small set of features that could explain the data effectively while not making the model unnecessarily complex. Filtering looks at one feature at a time and ignores how features work together, and stepwise selection may overlook useful combinations. So we used the most exhaustive wrapper approach, fitting logistic regression models to all 32,767 non-empty subsets of the 15 features. Each subset was evaluated with AIC, calculated as AIC=2k-2ln(L), where k is the number of model parameters and L is the maximized likelihood. Lower AIC is better, and 2k term penalized unnecessary features.

The ten best subsets had AIC values between 8947 and 8949. The subset with the lower AIC has three features, which were selected for the next modeling stage:

Table 2. Selected feature subset (lowest AIC).

#	Feature	Description
6	OMCLag1	Open minus close, lagged one day
8	OMCLag3	Open minus close, lagged three days
10	VolumeLag2	Trading volume, lagged two days

Interpretation. Two observations stand out from the selection results. First, the ten best subsets are within about two AIC points of each other, a difference generally considered too small to establish a meaningful distinction. This selected subset is a reasonable option, but not clearly the only good one. Second, I expected lagged closing prices to matter more, but the search chose intraday movement and volume. Although closing prices tend to persist over time, this persistence says little about whether the next return will be positive or negative.

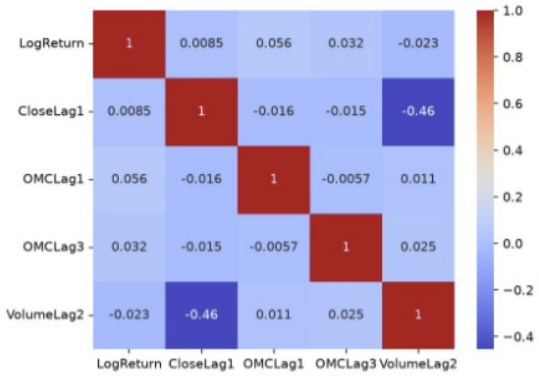


Figure 1. Correlation heat map of the selected features, with LogReturn and CloseLag1 included for comparison.

Figure 1 shows the difficulty of predicting daily returns. The correlations between each selected feature and LogReturn are all close to zero (OMCLag1 0.056, OMCLag3 0.032, VolumeLag2 -0.023) which suggests that none of the selected variables provides substantial linear information about the next return. The selected features are also nearly uncorrelated with each other, so each adds something separate. The strongest correlation in the heat map is -0.46 between CloseLag1 and VolumeLag2, probably because of the increase in Microsoft's share price while its share volume fell.

Research Design

Cross-validation. Because daily returns are ordered in time, shuffled k-fold cross-validation would risk allowing the model to learn from future observations while predicting earlier ones. We used Scikit-Learn's TimeSeriesSplit with five splits. Each split trains the model on everything before a block of data and tests on that block. We also introduced a 10-day gap between the training and test sets because the moving average features overlap in time and could carry information across the boundary. The test periods include several distinct market conditions (Table A1), including the 2008 financial crisis and the COVID-19 market decline in 2020.

Model and Tuning. We used XGBoost gradient boosting with the binary:logistic objective and a fixed random seed (2025). Compared with logistic regression, XGBoost models nonlinear relationships and interactions among features while also being a well-documented algorithm. Using default configurations and 1000 trees, the model achieved a mean cross-validated accuracy at 0.501 (std. dev. 0.021), essentially a coin flip and just below the 51.1% always-up benchmark. The best settings (Table A2) raised cross-validated accuracy to 0.521. That gain of about two points is roughly one standard deviation of the baseline's fold-to-fold variation, so it's a hint of improvement and not a clear one.

Model Evaluation

Of the 3,161 negative-return days, 2,192 were predicted correctly and 969 were not. Of the 3,307 positive-return days, 2,498 were predicted correctly and 809 were not. That's 4,690 out of 6,468 days, an in-sample accuracy of 72.5% (Figure 2). Between up and down days, the model tends to do somewhat better with the former (recall 0.76 vs recall 0.69). The ROC curve (Figure 3) has an AUC of 0.72. The most important result we found was with training data and time series cross-validation. On its training data the model classifies 72.5% correctly, but time series cross-validation showed about 52% on later data the model has not seen. A gap this large points to overfitting. The cross-validated 52% is the realistic expectation, and it's barely above the 51.1% benchmark.

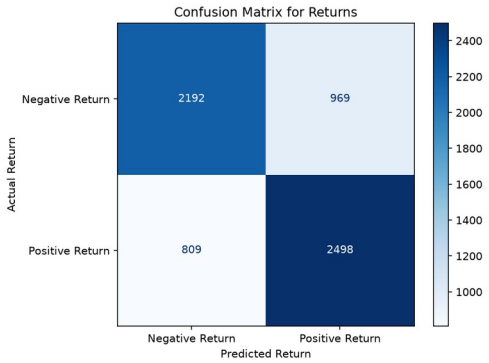


Figure 2. Confusion matrix for the final model (training data).

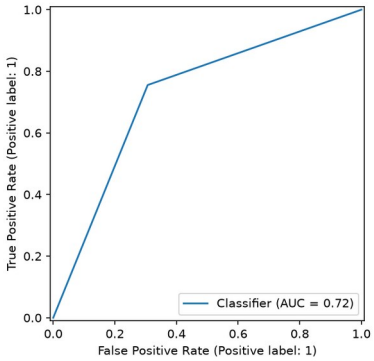


Figure 3. ROC curve for the final model (training data).

Conclusions and Next Steps

Like the jump-start code, I did not evaluate the final model on a hold-out test set, and the AIC search used the full sample. As a result, neither the feature selection nor the tuned model's score provides a fully reliable out-of-sample estimate. The search ranges as coded allow invalid subsample values above 1 as well. Using lagged open-minus-close and volume features with a tuned XGBoost, I could predict MSFT's next-day direction only slightly better than chance. This achieved an accuracy of about 52% in cross-validation compared with the 51% always-up benchmark. The 72.5% in-sample accuracy reflects overfitting, so careful validation matters more than model complexity with weak signal. Next steps are to hold out a true test set, draw the ROC curve from predicted probabilities, try a three-class target (up, stable, down), and add features beyond MSFT's price history. The code, figures, and dataset are available in the project repository: https://github.com/asenthilkumar1/MSDS-451-PA1

Appendix

Table A1. Time series cross-validation splits (5 splits, 10-day gap).

Split	Training rows	Test rows	Test period
1	1,068	1,078	2004-04 to 2008-08
2	2,146	1,078	2008-08 to 2012-11
3	3,224	1,078	2012-11 to 2017-02
4	4,302	1,078	2017-02 to 2021-06
5	5,380	1,078	2021-06 to 2025-09

Table A2. Best hyperparameters from randomized search (cross-validated accuracy 0.521).

Hyperparameter	Best value	What it does
max_depth	7	Tree depth; deeper trees are more complex
min_child_weight	2	Minimum weight in a leaf; lower is more complex
subsample	0.63	Share of rows used per tree; lower adds randomness
learning_rate	0.01	Size of each tree's contribution; lower resists overfitting
n_estimators	411	Number of boosting rounds